

“Airfoil Exploration and Performance Analysis using NASA Simulator”

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Winter Training Program on Space Science
Technology**

Subject Name: Advanced drone Technology

By

**Student Name: L. Harshita Bai
Institute Name: VIT AP University
Institute Roll no: 23BCE9648
Enrollment no: 974543**

**Under the Supervision of
Mr. Sandeep Kumar Khare**



**India Space Academy,
Department of Space Education, India Space Week**

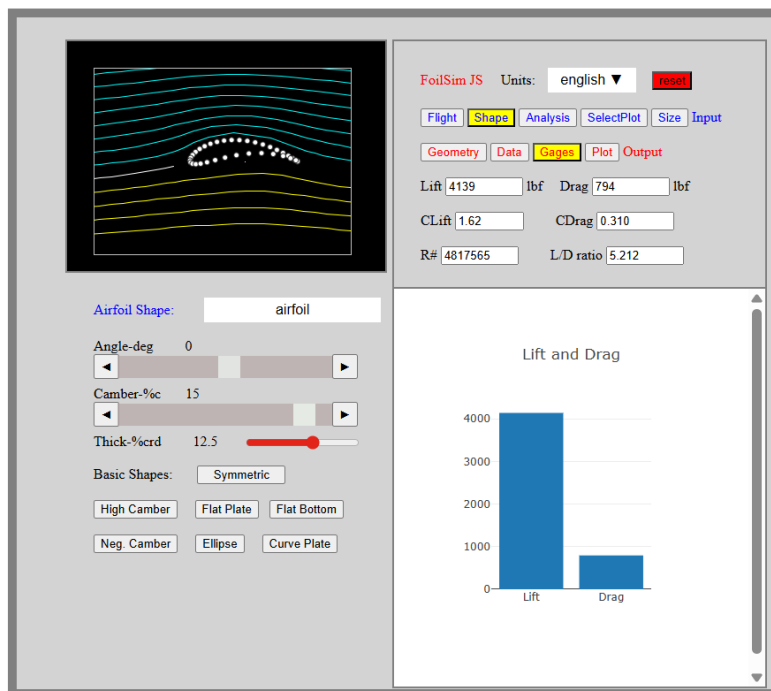
STEP 1 — Explore Default Airfoil

Question: Selecting a cambered airfoil and set Angle of Attack = 0° and observe the lift value? Does it generate lift? Why?

Answer: Lift value generated: 4139 lbf.

Yes, It has generated lift even when the Angle of Attack is 0° .

A cambered airfoil generates lift even at zero angle of attack because of its asymmetric curved shape. The curvature causes air to flow faster over the upper surface than the lower surface, resulting in lower pressure on the top and higher pressure below. This pressure difference produces an upward lift force. Additionally, the cambered airfoil deflects airflow downward, and according to Newton's third law, an equal and opposite upward force acts on the airfoil, generating lift even without any tilt.



STEP 2 — Study Effect of Angle of Attack

Question: Using the same cambered airfoil and Adjusting AoA to: 5° , 10° , 15° . Record: Lift value, Drag value (if available), Flow change (if shown).

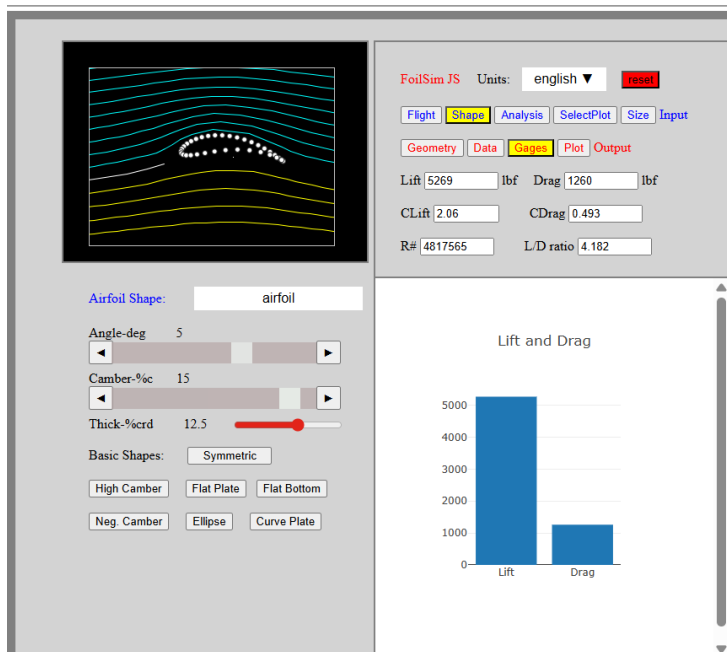
Answer:

a) **AOA to 5° :**

Lift value: 5269 lbf.

Drag value: 1260 lbf.

Flow change: Streamlines over the upper surface become more closely spaced, indicating increased airflow velocity and lift. A moderate downward deflection of airflow (downwash) is observed.

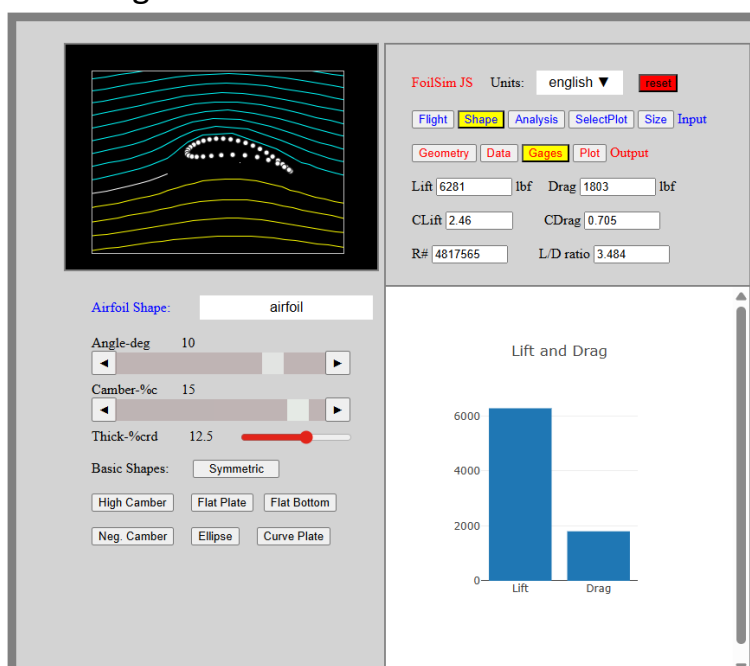


b) AOA at 10°:

Lift value: 6281 lbf.

Drag value: 1803 lbf.

Flow change: The streamlines show stronger curvature over the upper surface, indicating higher acceleration of airflow. Downwash increases significantly, producing higher lift. The flow is still mostly attached but shows signs of increased disturbance near the trailing edge.

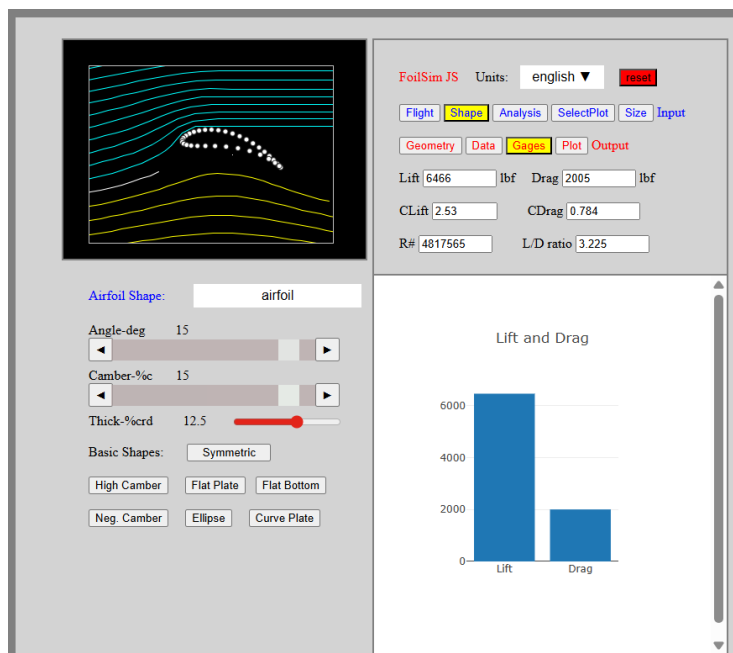


c) AOA at 15°:

Lift value: 6466 lbf.

Drag value: 2005 lbf.

Flow change: The airflow begins to lose smooth attachment over the upper surface. Streamlines appear distorted and less uniform, indicating the onset of flow separation. Lift growth slows down and the airfoil approaches stall conditions.



STEP 3 — Compare Airfoils

Question: Pick three airfoil types, e.g: Cambered, Symmetric, Flat Plate. For each airfoil: Repeat measurements at 0°, 5°, 10°, 15° Record lift (and drag if visible)?

Answer:

a) Cambered Airfoil:

AOA to 0°:

Lift value: 4139 lbf.

Drag value: 794 lbf.

AOA to 5°:

Lift value: 5269 lbf.

Drag value: 1260 lbf.

AOA at 10°:

Lift value: 6281 lbf.

Drag value: 1803 lbf.

AOA at 15°:

Lift value: 6466 lbf.

Drag value: 2005 lbf.

b) Symmetric Airfoil:

AOA to 0°:

Lift value: 0 lbf.

Drag value: 43 lbf.

AOA to 5°:

Lift value: 1492 lbf.

Drag value: 132 lbf.

AOA at 10°:

Lift value: 2842 lbf.

Drag value: 373 lbf.

AOA at 15°:

Lift value: 3609 lbf.

Drag value: 678 lbf.

c) Flat Plate:

AOA to 0°:

Lift value: 0 lbf.

Drag value: 23 lbf.

AOA to 5°:

Lift value: 1342 lbf.

Drag value: 122 lbf.

AOA at 10°:

Lift value: 2568 lbf.

Drag value: 384 lbf.

AOA at 15°:

Lift value: 3269 lbf.

Drag value: 672 lbf.

STEP 4 — Analyze

1. Which airfoil produced highest lift at moderate angle (5°–10°)?

Answer: The cambered airfoil produced the highest lift at moderate angles of attack. At 5°, it generated 5269 lbf, and at 10°, it produced 6281 lbf, which is significantly higher than the symmetric and flat plate airfoils. This is due to its curved shape, which enhances pressure difference and lift generation.

2. Which airfoil stalled first (if lift dropped at 15°)?

Answer: None of the airfoils showed a clear lift drop at 15° , but the cambered airfoil showed only a small increase in lift from 6281 lbf (10°) to 6466 lbf (15°), indicating that it is approaching stall. This suggests that the cambered airfoil would stall earlier than the symmetric and flat plate airfoils.

3. Why does a symmetrical airfoil show zero lift at 0° AoA?

Answer: A symmetrical airfoil has identical upper and lower surfaces. At 0° angle of attack, airflow velocity and pressure are the same on both sides of the airfoil, resulting in no pressure difference. Hence, no lift is generated.

4. Why do cambered airfoils make lift even with no tilt?

Answer: Cambered airfoils have an asymmetric curved shape that accelerates airflow over the upper surface even at 0° AoA. This creates lower pressure on top and higher pressure below, producing lift. Additionally, the airflow is deflected downward, generating lift according to Newton's third law.

5. Which airfoil would you choose for:

Fixed-wing UAV (long endurance)

Racing drone propeller

Glider

Answer: a) Fixed-wing UAV (Long Endurance)

A cambered airfoil is preferred because it produces high lift at low angles of attack and low speeds, improving endurance and efficiency during long-duration flights.

b) Racing Drone Propeller

A symmetric airfoil is suitable for racing drone propellers because it provides balanced performance at high rotational speeds and works efficiently in both directions of airflow.

c) Glider

A cambered airfoil is ideal for gliders because it generates high lift with minimal power, allowing sustained flight and efficient soaring.

“Design and Implementation of a Drone Telemetry Navigation System”

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Winter Training Program on Space Science
Technology**

Subject Name: Advanced drone Technology

By

Student Name: L. Harshita Bai

Institute Name: VIT AP University

Institute Roll no: 23BCE9648

Enrollment no: 974543

Under the Supervision of
Mr. Sandeep Kumar Khare



**India Space Academy,
Department of Space Education, India Space Week**

Object Detection for Rescue Operations.

Scenario: You are developing an AI for a search-and-rescue drone. The drone needs to scan a forest and automatically put a "Target Lock" on specific objects (like people or vehicles) using the YOLOv8 neural network.

Code:

```
import cv2
import numpy as np
import time
from ultralytics import YOLO
# Load YOLOv8 model
model = YOLO("yolov8n.pt")
def process_drone_feed(source=0):
    cap = cv2.VideoCapture(source)
    prev_time = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        frame_resized = cv2.resize(frame, (640, 640))
        # Run YOLO
        results = model(frame_resized)[0]
        if results.boxes is not None:
            for box in results.boxes:
                x1, y1, x2, y2 = map(int, box.xyxy[0])
                conf = float(box.conf[0])
                cls = int(box.cls[0])
                # LOWER CONF THRESHOLD
                if conf < 0.4:
                    continue
                # Center logic
                center_x = (x1 + x2) / 2
                if 640*0.4 < center_x < 640*0.6:
                    color = (0,0,255)
                    status = "TARGET LOCK"
                else:
                    color = (0,255,0)
```



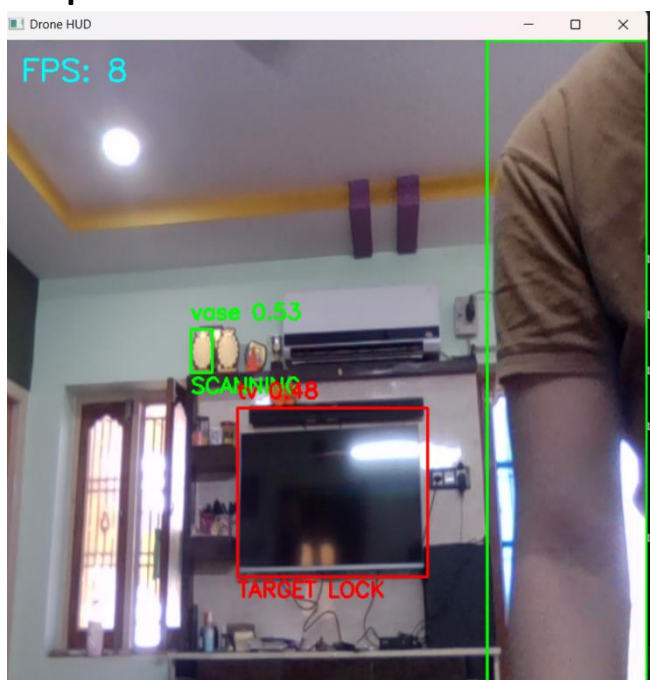
```

        status = "SCANNING"
        label = model.names[cls]
        # Draw
        cv2.rectangle(frame_resized,(x1,y1),(x2,y2),color,2)
        cv2.putText(frame_resized,f"{label} {conf:.2f}",
                    (x1,y1-10),
                    cv2.FONT_HERSHEY_SIMPLEX,0.7,color,2)
        cv2.putText(frame_resized,status,
                    (x1,y2+20),
                    cv2.FONT_HERSHEY_SIMPLEX,0.7,color,2)

    # FPS
    curr_time = time.time()
    fps = 1/(curr_time-prev_time) if prev_time else 0
    prev_time = curr_time
    cv2.putText(frame_resized,f"FPS: {int(fps)}",
                (20,40),
                cv2.FONT_HERSHEY_SIMPLEX,1,(255,255,0),2)
    cv2.imshow("Drone HUD", frame_resized)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
    cap.release()
    cv2.destroyAllWindows()
process_drone_feed(0)

```

Output:





“Computer Vision Techniques using OpenCV and NumPy for Drone Applications”

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Winter Training Program on Space Science
Technology**

Subject Name: Advanced drone Technology

By

Student Name: L. Harshita Bai

Institute Name: VIT AP University

Institute Roll no: 23BCE9648

Enrollment no: 974543

Under the Supervision of

Mr. Sandeep Kumar Khare



**India Space Academy,
Department of Space Education, India Space Week**

Task 1 – Landing Pad Finder

Code:

```
import cv2

import numpy as np

print("Task 1: Landing Pad Finder Running...")

# Load image

img = cv2.imread("landing_pad.jpg")

# Check if image loaded properly

if img is None:

    print("Error: Image not found!")

    exit()

# Make a copy for drawing

output = img.copy()

# Convert to grayscale

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply blur to reduce noise

gray = cv2.GaussianBlur(gray, (9, 9), 2)

# Detect circles

circles = cv2.HoughCircles(

    gray,

    cv2.HOUGH_GRADIENT,

    dp=1.2,

    minDist=50,

    param1=100,

    param2=30,

    minRadius=50,

    maxRadius=300

)
```

```

# If circles detected
if circles is not None:
    circles = np.round(circles[0, :]).astype("int")
    for (x, y, r) in circles:
        # Draw outer circle
        cv2.circle(output, (x, y), r, (0, 255, 0), 2)

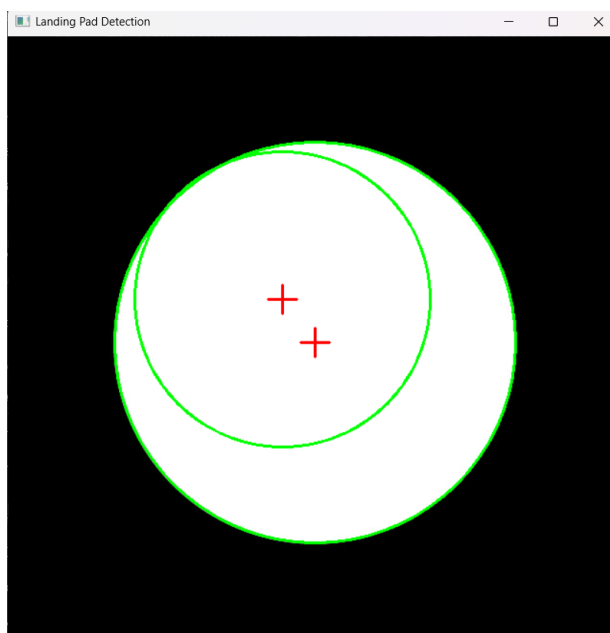
        # Draw crosshair
        cv2.line(output, (x - 15, y), (x + 15, y), (0, 0, 255), 2)
        cv2.line(output, (x, y - 15), (x, y + 15), (0, 0, 255), 2)

        print("Target Acquired at center:", (x, y))
else:
    print("No circles detected.")

# Show result
cv2.imshow("Landing Pad Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Output:



C:\Users\L.Vinay\OneDrive\Desktop\CV_Assignment>python cv_assignment.py

Task 1: Landing Pad Finder Running...

Target Acquired at center: (np.int64(320), np.int64(319))

Target Acquired at center: (np.int64(286), np.int64(274))

Explanation:

The image was loaded using cv2.imread() and converted to grayscale for processing. Gaussian blur was applied to reduce noise. The Hough Circle Transform was used to detect circular shapes in the image. NumPy was used to round the detected coordinates to integers for accurate drawing. A crosshair was drawn at the detected center to indicate the landing pad location.

Task 2 – Horizon Leveler (Image Rotation)

Code:

```
import cv2

import numpy as np

print("Task 2: Horizon Leveler Running...")

# Load image
img = cv2.imread("landing_pad.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

def level_horizon(image, angle):
    # Get image dimensions
    (h, w) = image.shape[:2]
    center = (w // 2, h // 2)

    # Create rotation matrix
    M = cv2.getRotationMatrix2D(center, angle, 1.0)

    # Rotate image
    rotated = cv2.warpAffine(image, M, (w, h))

    # Crop black edges using NumPy slicing
    crop_x = int(w * 0.1)
    crop_y = int(h * 0.1)
    cropped = rotated[crop_y:h - crop_y, crop_x:w - crop_x]
```

```

    return cropped

# Rotate image by -15 degrees
fixed_image = level_horizon(img, -15)

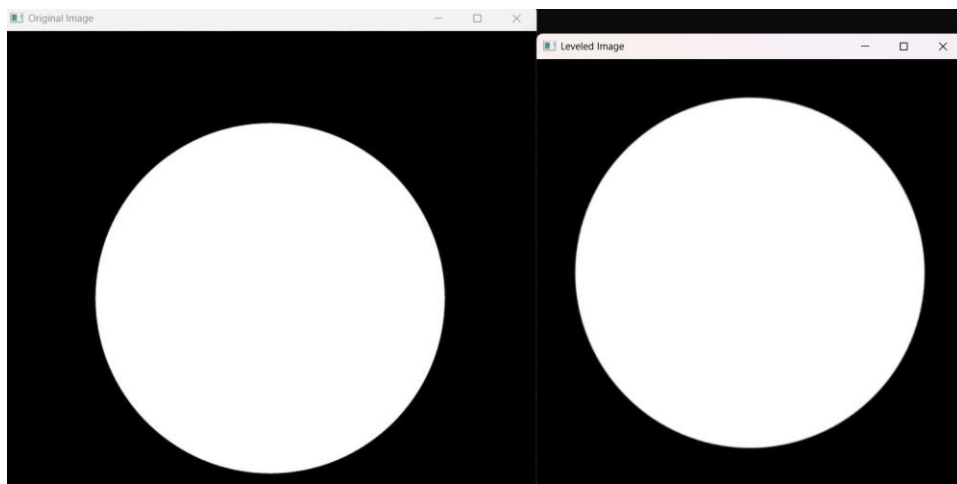
# Show result
cv2.imshow("Original Image", img)
cv2.imshow("Leveled Image", fixed_image)

cv2.waitKey(0)

cv2.destroyAllWindows()

```

Output:



Explanation:

A rotation matrix was created using `cv2.getRotationMatrix2D()`. The image was rotated using `cv2.warpAffine()` to correct the tilt. Since rotation introduces black edges, NumPy slicing was used to crop the central portion of the image. This avoids using loops and ensures efficient processing.

Task 3 – Obstacle Alert System (Color Thresholding).

Code:

```

import cv2

import numpy as np

print("Creating test red image...")

# Create black image

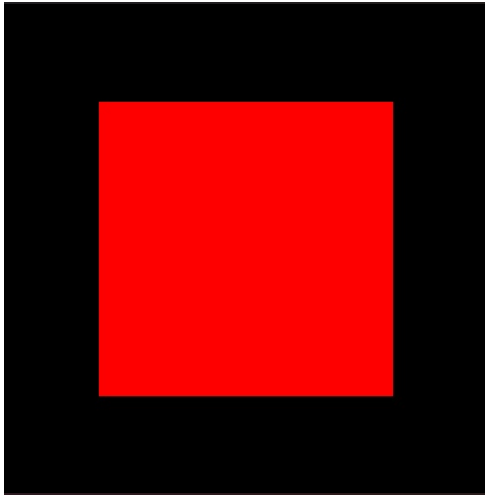
img = np.zeros((500, 500, 3), dtype=np.uint8)

# Draw red rectangle (BGR format: (Blue, Green, Red))

```

```
cv2.rectangle(img, (100, 100), (400, 400), (0, 0, 255), -1)
cv2.imwrite("red_test.jpg", img)
print("Red test image created!")
```

Output:



```
C:\Users\L.Vinay\OneDrive\Desktop\CV_Assignment>python cv_assignment.py
```

```
Creating test red image...
```

```
Red test image created!
```

Code:

```
import cv2
import numpy as np
print("Task 3: Obstacle Alert System Running...")
# Load image
img = cv2.imread("red_test.jpg")
if img is None:
    print("Error: Image not found!")
    exit()
# Convert BGR to HSV
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
# Red has two HSV ranges
lower_red1 = np.array([0, 120, 70])
upper_red1 = np.array([10, 255, 255])
```



```

lower_red2 = np.array([170, 120, 70])
upper_red2 = np.array([180, 255, 255])

# Create masks
mask1 = cv2.inRange(hsv, lower_red1, upper_red1)
mask2 = cv2.inRange(hsv, lower_red2, upper_red2)
mask = mask1 + mask2

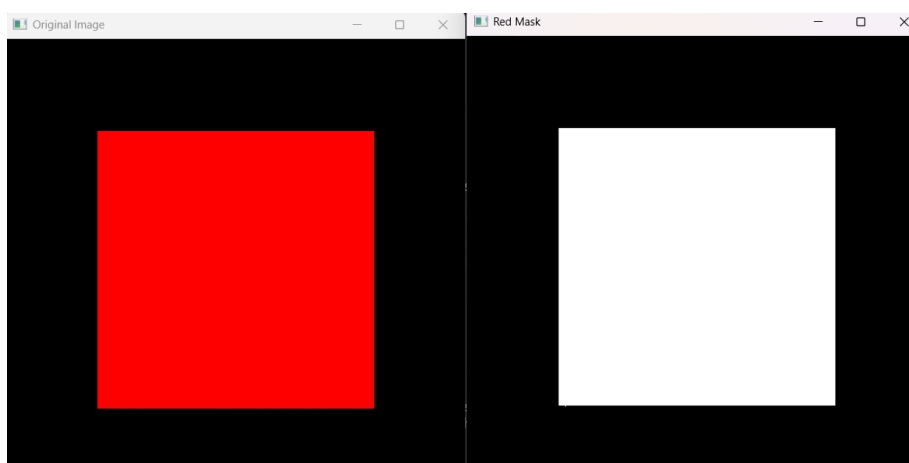
# Calculate danger percentage
red_pixels = np.count_nonzero(mask)
total_pixels = mask.size
danger_percentage = (red_pixels / total_pixels) * 100
print("Danger Percentage:", danger_percentage)

if danger_percentage > 10:
    print("DANGER: OBSTACLE DETECTED!")

# Show mask
cv2.imshow("Original Image", img)
cv2.imshow("Red Mask", mask)
cv2.waitKey(0)
cv2.destroyAllWindows()

```

Output:



C:\Users\L.Vinay\OneDrive\Desktop\CV_Assignment>python cv_assignment.py

Task 3: Obstacle Alert System Running...

Danger Percentage: 36.2408

DANGER: OBSTACLE DETECTED!

Explanation:

The image was converted from BGR to HSV color space because red is easier to detect in HSV. Two HSV ranges were used to capture red color. The mask was created using `cv2.inRange()`. NumPy's `np.count_nonzero()` was used to calculate the percentage of red pixels. If more than 10% of the image is red, a danger warning is printed.

Task 4 – Night Vision Booster

Code:

```
import cv2

import numpy as np

print("Task 4: Night Vision Booster Running...")

# Load image

img = cv2.imread("red_test.jpg")

if img is None:

    print("Error: Image not found!")

    exit()

# Increase brightness safely using NumPy

enhanced_img = np.clip(img + 80, 0, 255).astype(np.uint8)

# Show images

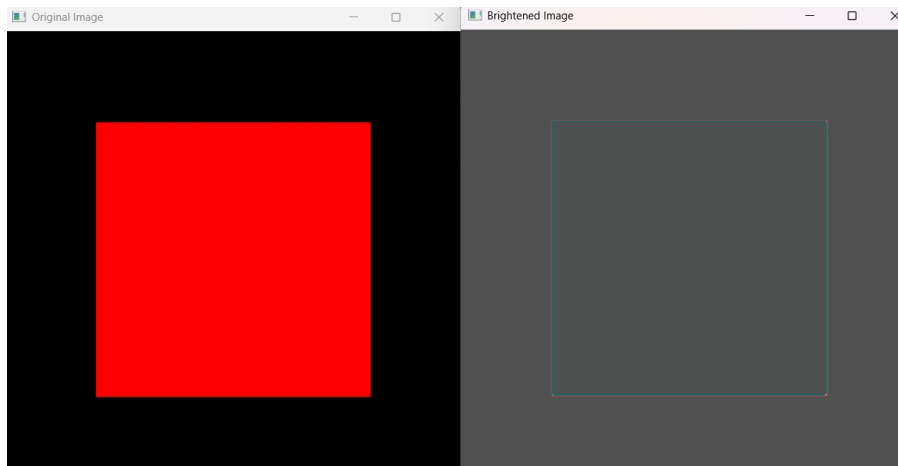
cv2.imshow("Original Image", img)

cv2.imshow("Brightened Image", enhanced_img)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Output:



```
C:\Users\L.Vinay\OneDrive\Desktop\CV_Assignment>python cv_assignment.py
```

Task 4: Night Vision Booster Running...

Explanation:

The brightness of the image was increased by adding a constant value to all pixel values using NumPy. Since adding values can cause overflow beyond 255, `np.clip()` was used to limit the pixel range between 0 and 255. The result was converted back to `uint8` format for proper display. This method uses vectorized operations and avoids loops, making it efficient.

Task 5 – Motion Blur Simulator Code

Code:

```
import cv2

import numpy as np

print("Task 5: Motion Blur Simulator Running...")

# Load image
img = cv2.imread("red_test.jpg")

if img is None:
    print("Error: Image not found!")
    exit()

# Create 5x5 kernel filled with zeros
kernel = np.zeros((5, 5))

# Place 0.2 in every cell of the middle row
kernel[2, :] = 0.2

print("Kernel Matrix:")
```

```
print(kernel)

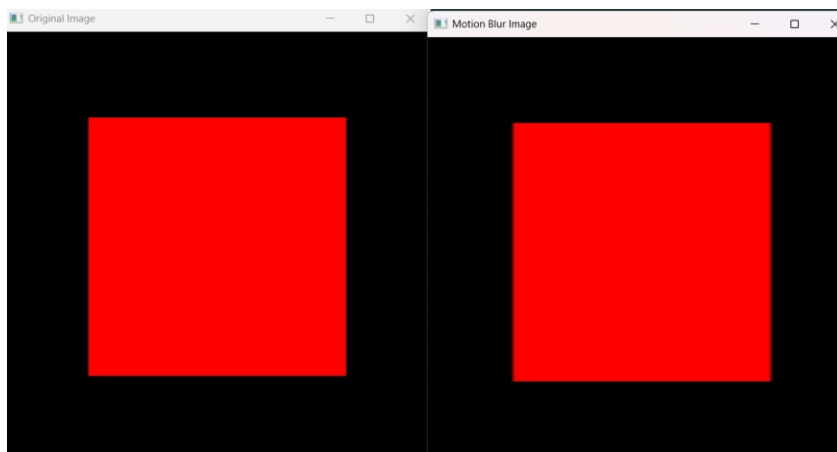
# Apply motion blur using convolution
motion_blur = cv2.filter2D(img, -1, kernel)

# Show images
cv2.imshow("Original Image", img)
cv2.imshow("Motion Blur Image", motion_blur)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

Output:



```
C:\Users\L.Vinay\OneDrive\Desktop\CV_Assignment>python cv_assignment.py
```

Task 5: Motion Blur Simulator Running...

Kernel Matrix:

```
[[0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  0. ]
 [0.2 0.2 0.2 0.2 0.2]
 [0.  0.  0.  0.  0. ]
 [0.  0.  0.  0.  0. ]]
```

Explanation:

In this task, a custom 5×5 kernel matrix was created using NumPy. The middle row was filled with 0.2 values so that the total sum of the kernel equals 1.0, ensuring that image brightness remains unchanged. This kernel was applied using `cv2.filter2D()` to simulate horizontal motion blur. The blur effect spreads pixel values across the row, creating a motion effect without using loops.

**“Control System Design and Analysis of a VTOL Aircraft”
Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Winter Training Program on Space Science
Technology**

Subject Name: Advanced drone Technology

By

Student Name: L. Harshita Bai
Institute Name: VIT AP University
Institute Roll no: 23BCE9648
Enrollment no: 974543

**Under the Supervision of
Mr. Sandeep Kumar Khare**



**India Space Academy,
Department of Space Education, India Space Week**

1. Introduction

For this project, I have selected a QuadPlane VTOL drone that uses a Pixhawk flight controller with ArduPilot firmware. A QuadPlane is a special type of unmanned aerial vehicle that combines the features of a quadcopter and a fixed-wing airplane. It can take off and land vertically using its four vertical motors, and once it gains sufficient speed, it can fly forward like a normal airplane using its wings.

VTOL drones require a special control system because they do not operate in just one flight mode. During take-off and hover, the drone behaves like a multirotor, where stability is maintained by adjusting the speed of individual motors. However, during forward flight, lift is mainly produced by the wings, and control is achieved using control surfaces such as ailerons and elevators. Since these two modes of flight are very different, the control system must be designed carefully to handle both.

The most challenging part of a VTOL system is the transition between vertical and forward flight. During this phase, the drone must gradually shift from motor-based control to aerodynamic control without losing stability. The control system ensures that the aircraft remains balanced, responds correctly to pilot commands, and performs a smooth and safe transition. Therefore, the flight controller and sensors play a very important role in the successful operation of a QuadPlane VTOL.

2. Overview of the Control System

The control system of the selected QuadPlane VTOL is built around the Pixhawk flight controller, which acts as the main processing unit of the drone. The Pixhawk is a compact electronic board that connects to all the sensors, motors, and control surfaces of the aircraft. It continuously receives data from sensors such as the IMU, GPS, and magnetometer, processes this information, and then sends appropriate signals to the motors and servos to maintain stability.

The Pixhawk flight controller runs on ArduPilot firmware. ArduPilot is an open-source autopilot software that supports multiple types of aircraft, including multirotors, fixed-wing airplanes, and hybrid VTOL systems. In the case of a QuadPlane, ArduPilot combines multirotor control logic and fixed-wing control logic within the same system. This allows the aircraft to operate in vertical take-off mode, transition mode, and forward flight mode without needing separate controllers.

The reason Pixhawk with ArduPilot is suitable for VTOL operations is because it supports advanced flight modes and automatic transition control. It includes built-in control algorithms that adjust motor speeds during hover and manage control surfaces during forward flight. The firmware also allows smooth switching between flight modes by gradually shifting control authority from vertical motors to aerodynamic surfaces.

Another important feature of this control system is its feedback mechanism. The controller constantly compares the desired position or attitude with the actual measured values from

sensors. If any difference is detected, it immediately corrects the error by adjusting motor outputs or control surfaces. This continuous feedback loop ensures stable and safe flight in all phases of operation.

Overall, the Pixhawk and ArduPilot combination provides a flexible and reliable platform for controlling hybrid VTOL aircraft like the QuadPlane.

Basic VTOL Control System Block Diagram

[Sensors]

(IMU, GPS, Magnetometer, Airspeed)



[Pixhawk Flight Controller]

(ArduPilot Firmware)



[Actuators]

(Motors + Control Surfaces)



[Drone Motion]



(Feedback to Sensors)

3. Sensors Used for Control

In a QuadPlane VTOL system, sensors play a very important role because the flight controller depends completely on sensor data to maintain stability and navigation. Without accurate sensor inputs, the aircraft cannot balance itself or perform smooth transitions.

3.1 Inertial Measurement Unit (IMU)

The IMU consists of a gyroscope and an accelerometer. The gyroscope measures the angular rotation of the drone around the roll, pitch, and yaw axes. The accelerometer measures linear acceleration in different directions.

The IMU is the most important sensor for flight stability. During vertical take-off and hover, the controller constantly checks IMU data to keep the aircraft balanced. If the drone tilts slightly, the controller immediately adjusts motor speeds to correct the tilt. Even during forward flight, the IMU helps maintain proper orientation and prevents instability.

3.2 Global Positioning System (GPS)

The GPS sensor provides information about the drone's position, altitude, and ground speed. It allows the aircraft to know its location relative to the Earth.

GPS is important for navigation and autonomous flight. It enables functions such as waypoint navigation, return-to-launch, and position hold. In VTOL systems, GPS is especially useful during hover and landing to maintain stable position.

3.3 Magnetometer

The magnetometer measures the Earth's magnetic field and helps determine the heading direction of the aircraft.

This sensor is important for maintaining correct yaw orientation. Without a magnetometer, the drone may drift or rotate unintentionally. During transition and cruise flight, correct heading information is necessary for proper navigation.

3.4 Airspeed Sensor

The airspeed sensor measures the speed of air flowing over the aircraft. This is different from ground speed measured by GPS.

The airspeed sensor becomes very important during transition and forward flight. It ensures that the aircraft has enough forward speed to generate lift from the wings. If airspeed is too low, the aircraft may stall. Therefore, the control system monitors airspeed before fully switching from vertical motors to wing-based lift.

4. Control in Different Flight Modes

4.1 Vertical Take-Off and Hover

During vertical take-off and hover, the QuadPlane behaves like a normal quadcopter. In this mode, only the four vertical lift motors are active. The forward thrust motor and control surfaces are either inactive or play a very small role.

Lift is generated completely by the vertical propellers. The flight controller adjusts the speed of each motor to maintain balance and stability.

- Roll control is achieved by increasing the speed of motors on one side and decreasing speed on the opposite side.
- Pitch control is managed by changing the speed of the front and rear motors.
- Yaw control is controlled by the difference in rotational torque between clockwise and counterclockwise rotating motors.

The control system works using feedback loops. The IMU continuously measures orientation, and the controller compares the desired angle with the actual angle. If any deviation is detected, the motor speeds are corrected immediately. This allows stable hover even in the presence of small disturbances like wind.

In this mode, the aircraft mainly uses multirotor control logic.

4.2 Transition Phase

The transition phase is the most critical and complex part of VTOL operation. During this phase, the aircraft gradually shifts from vertical lift to forward wing-based lift.

At the beginning of transition:

- The forward thrust motor starts increasing speed.
- The aircraft begins tilting slightly forward.
- Airspeed gradually increases.

As forward speed builds up, the wings start generating lift. The flight controller then slowly reduces the power of the vertical lift motors. Instead of an instant switch, the system blends both control methods to avoid instability.

During this stage, the control system must carefully monitor airspeed and attitude. If airspeed is insufficient, the aircraft may lose lift. Therefore, the controller ensures that a safe airspeed is reached before completely switching to fixed-wing mode.

Major challenges during transition include:

- Sudden loss of lift
- Aerodynamic instability
- Incorrect pitch angle
- Wind disturbances

The ArduPilot firmware handles this using predefined transition logic and safety checks.

4.3 Forward (Cruise) Flight

Once the transition is complete, the QuadPlane operates like a conventional fixed-wing aircraft.

In this mode:

- Vertical lift motors are turned off or idle.
- The wings generate lift.
- The forward motor provides thrust.
- Control surfaces handle orientation.

Control is achieved using:

- Ailerons for roll
- Elevator for pitch
- Rudder for yaw

Unlike hover mode, where stability is maintained by varying motor speeds, cruise flight relies on aerodynamic forces. The controller now uses fixed-wing control algorithms instead of multirotor logic.

Cruise flight is more energy efficient compared to hover mode because the wings carry most of the aircraft weight.

Flight Mode Transition in QuadPlane VTOL

Vertical Take-Off & Hover



Transition Phase

(Blending of motor and wing control)



Forward Cruise Flight

5. Control Mixing and Logic

Control mixing refers to the method by which the flight controller distributes control commands to different actuators such as motors and control surfaces. In a hybrid VTOL system like the QuadPlane, control mixing is especially important because the aircraft has two different control mechanisms: motor-based control and aerodynamic surface control.

In hover mode, the flight controller uses multirotor mixing logic. For example, if a pitch command is given, the controller increases the speed of the rear motors and decreases the speed of the front motors to tilt the aircraft forward. Similarly, roll and yaw commands are achieved by adjusting motor speeds accordingly.

However, during forward flight, the control logic changes completely. Instead of adjusting motor speeds for orientation control, the controller uses aerodynamic control surfaces. A pitch command moves the elevator, a roll command moves the ailerons, and a yaw command moves the rudder. The forward motor mainly provides thrust and does not control attitude like in hover mode.

During transition, the control system blends both control methods. The flight controller gradually reduces the influence of motor-based control and increases the influence of control surfaces. This blending process ensures that the aircraft does not experience sudden instability when switching between flight modes.

The flight controller decides what to control based on the active flight mode selected in the firmware. ArduPilot contains predefined logic that automatically changes the control mixing strategy depending on whether the drone is in hover, transition, or cruise mode. This intelligent switching makes hybrid VTOL flight possible and stable.

6. Fail-Safe and Stability Control

In a VTOL system, safety is extremely important because the aircraft operates in multiple flight modes. The flight controller includes several fail-safe mechanisms to prevent accidents in case of sensor failure or unstable transition.

If the GPS signal fails, the aircraft cannot accurately determine its position. In such cases, the system may switch from position-hold mode to altitude-hold or manual mode, depending on configuration. The IMU still maintains attitude stability even if GPS data is lost.

If the transition process fails, for example if the required airspeed is not achieved, the control system can abort the transition and return to hover mode. This prevents the aircraft from stalling or losing lift.

Stability is maintained using continuous feedback control loops. The controller constantly compares desired flight parameters such as attitude, altitude, and speed with actual measured values from sensors. If any error is detected, corrective signals are immediately sent to motors or control surfaces. This rapid correction helps the aircraft remain stable even under wind disturbances or sudden changes in load.

Overall, the fail-safe system ensures that the QuadPlane can handle unexpected situations safely.

7. Key Challenges in VTOL Control Systems

Although VTOL drones provide flexibility and efficiency, their control systems face several technical challenges.

One major challenge is transition instability. During the shift from vertical to forward flight, lift source changes from propellers to wings. If this shift is not managed properly, the aircraft can lose balance or stall.

Another challenge is control tuning complexity. The controller must be tuned for both multirotor and fixed-wing behavior within the same aircraft. Improper tuning can lead to oscillations or poor stability.

Power imbalance during hover is also a concern. Since vertical lift motors carry the full weight of the aircraft during take-off and landing, any imbalance in thrust can affect stability.

Sensor dependency is another challenge. If sensor data is inaccurate or delayed, the control system may make incorrect adjustments.

Finally, environmental disturbances such as wind gusts can significantly affect transition and landing phases.

Managing all these challenges requires careful design of control algorithms and continuous feedback monitoring.

8. Conclusion

Through this case study of the QuadPlane VTOL using Pixhawk and ArduPilot firmware, it is clear that the control system is the most critical component of a hybrid aircraft. Since the drone operates in both multirotor and fixed-wing modes, the controller must manage two completely different flight principles within the same system.

The most challenging phase of operation is the transition between vertical and forward flight. During this phase, the aircraft shifts from motor-based lift to aerodynamic lift generated by the wings. A well-designed control system ensures that this transition happens smoothly without loss of stability.

The integration of sensors such as the IMU, GPS, magnetometer, and airspeed sensor allows the flight controller to continuously monitor the aircraft's condition. By using feedback loops and intelligent control mixing, the system maintains stability in hover, transition, and cruise modes.

This study helped in understanding how complex and carefully engineered VTOL control systems are. It shows that without advanced control logic and reliable sensor feedback, safe operation of hybrid drones would not be possible.